

Mundo Pokémon — Fase 3: renderizado dinámico del listado con JavaScript

Propósito de esta fase

En esta fase vas a dar el salto desde una maqueta estática a una interfaz que **se construye a partir de datos**. El objetivo no es añadir todavía búsqueda, formularios o peticiones a APIs, sino aprender a **pintar el listado de Pokémon dinámicamente con JavaScript** usando una colección local de datos.

Qué se espera que sepas hacer al terminar

Al finalizar esta fase deberías ser capaz de:

- definir una colección local de objetos en JavaScript;
- recorrer esa colección;
- crear tarjetas dinámicamente en el DOM;
- insertar contenido textual, imágenes y clases según los datos;
- separar razonablemente los datos de la lógica de renderizado;
- comprender que la interfaz puede generarse desde la información disponible.

Carácter de la fase

Esta fase forma parte de la **práctica del proyecto**.

Aquí vas a aplicar los contenidos de la sesión sobre **DOM y renderizado dinámico** en un contexto más realista.

1. Contexto de la fase

Hasta ahora, la interfaz de Mundo Pokémon podía estar construida de forma estática en HTML y CSS. En esta fase, el objetivo es que la parte del **listado principal** deje de depender de tarjetas escritas manualmente y pase a generarse desde JavaScript.

Vas a trabajar con una colección local de Pokémon definida en tu archivo `.js`. A partir de esos datos, deberás renderizar las tarjetas dentro del contenedor correspondiente del listado.

Idea central

En esta fase no importa todavía de dónde vienen los datos “de verdad”. Lo importante es que aprendas a construir la interfaz **a partir de una estructura de datos**.

2. Objetivo funcional

Debes conseguir que la zona del listado de Mundo Pokémon:

- muestre varias tarjetas generadas con JavaScript;
- tome la información de un array de objetos;
- se rellene dinámicamente al cargar la página;
- mantenga una estructura visual coherente con la maqueta que ya estés construyendo.

3. Qué debes construir

3.1. Una colección local de Pokémon

Debes definir en JavaScript un **array de objetos**.

Cada objeto representará un Pokémon.

Te toca decidir qué información necesita cada tarjeta y, por tanto, qué propiedades debe tener cada objeto. Para hacerlo bien, revisa con atención la tarjeta que ya tienes maquetada y piensa qué datos visibles necesita realmente.

Por ejemplo, necesitarás propiedades como: `id`, `nombre`, ...

Relación entre datos y maqueta

La estructura del objeto Pokémon debe corresponderse con la información visible que necesita la tarjeta.

Si una pieza de información aparece en la interfaz, debes plantearte de dónde saldrá en el objeto.

🕒 [TODO] Adelanto de investigación

En próximas iteraciones de este proyecto estos datos ya no serán una colección local, sino que los tendrás que obtener de manera externa, concretamente desde la **PokéAPI**.

Una vez que tengas claro el listado de propiedades que te hace falta para este ejercicio, te invito a que vayas investigando de dónde puedes sacar cada uno de esos datos.

Puedes consultar la documentación en:

<https://pokeapi.co/docs/v2>

3.2. Un contenedor para el listado

Debes tener en el HTML una zona específica donde se insertarán las tarjetas. Si hiciste el maquetado en la fase anterior, deberías poder identificarla con claridad.

Ese contenedor debe ser:

- único;
- claramente reconocible en el HTML;
- y fácilmente seleccionable desde JavaScript.

Ese será el destino del renderizado dinámico.

3.3. Una función que cree una tarjeta

Debes crear una función que reciba un objeto Pokémon y devuelva el nodo DOM correspondiente a su tarjeta.

Esa función debería encargarse de:

- crear la estructura principal de la tarjeta;
- insertar nombre, tipo, imagen y toda la información que se necesite;
- añadir clases CSS;
- devolver una única tarjeta completamente construida.

Importante

La función debe construir **una sola tarjeta**, no renderizar toda la colección.

Idea clave

Una función que crea una tarjeta trabaja con **un solo Pokémon**.

La función que renderiza la colección trabaja con **el array completo**.

3.4. Una función que renderice el listado completo

Debes crear una segunda función que:

- reciba el array completo;
- vacíe el contenedor si es necesario;
- recorra la colección;
- cree cada tarjeta;
- inserte todas las tarjetas en el DOM.

Recomendación técnica

Es recomendable usar un `DocumentFragment` para preparar las inserciones antes de añadirlas al contenedor final.

4. Requisitos obligatorios

Tu solución debe cumplir, como mínimo, estas condiciones:

4.1. Datos

- usar un **array de objetos**;
- incluir al menos **9 Pokémon**;
- mantener una estructura de propiedades coherente en todos ellos.

4.2. DOM

- seleccionar correctamente el contenedor del listado;
- crear las tarjetas con `createElement()` ;
- insertar el contenido textual con `textContent()` ;
- insertar atributos como la imagen con `setAttribute()` o propiedades equivalentes;
- añadir las tarjetas al DOM con `append()` .

4.3. Organización

- usar una función para crear tarjetas;
- usar una función para renderizar la colección;
- evitar resolver toda la fase en un único bloque de código desordenado.

4.4. Estilo de construcción

- no basar la solución exclusivamente en `innerHTML` ;
 - mantener nombres claros de variables y funciones;
 - procurar que la estructura del código sea legible.
-

5. Qué parte de la interfaz debe resolverse en esta fase

Obligatorio en esta fase

- el **listado general de tarjetas**;
- el renderizado dinámico desde datos locales;
- la estructura básica visual de cada Pokémon.

Todavía no es obligatorio en esta fase

- formularios;
- caja de búsqueda funcional;
- filtros dinámicos;
- eventos complejos;
- consumo de API;
- persistencia;
- asincronía.

No te adelantes

El objetivo de esta fase no es resolver todo el proyecto de una vez, sino consolidar muy bien el patrón:

datos locales → **función de tarjeta** → **función de renderizado** → **listado visible**.

6. Secuencia de trabajo recomendada

Paso 1. Revisa que tengas preparado el HTML base

Asegúrate de tener:

- la estructura principal de la página;
- el contenedor del listado;
- los estilos mínimos para que las tarjetas se distingan.

Paso 2. Define la colección local

Crea el array de objetos Pokémon en tu archivo JavaScript.

Paso 3. Selecciona el contenedor

Comprueba que lo estás localizando correctamente con una función de selección adecuada y que el nodo obtenido es exactamente el contenedor donde deben insertarse las tarjetas.

Paso 4. Construye una sola tarjeta

Antes de intentar renderizar toda la colección, crea una tarjeta manualmente desde JavaScript para comprobar que la estructura visual funciona.

Paso 5. Convierte esa lógica en una función reutilizable

Haz que esa función reciba un Pokémon y devuelva la tarjeta.

Paso 6. Recorre la colección completa

Crea la función de renderizado y pinta todas las tarjetas en el contenedor.

7. Criterios de calidad

Tu solución será mejor si:

- el array está bien modelado;
- las propiedades tienen nombres claros;
- la función que crea la tarjeta tiene una responsabilidad concreta;
- la función de renderizado no contiene lógica innecesaria;
- el HTML generado mantiene una estructura limpia;
- el CSS se aprovecha mediante clases bien asignadas;

- la interfaz final es legible y coherente.
-

8. Errores frecuentes que debes evitar

 Errores muy habituales en esta fase

8.1. Escribir las tarjetas directamente en el HTML

Eso impediría practicar el objetivo real de la fase, que es construir el listado desde JavaScript.

8.2. Usar un objeto suelto cuando necesitas una colección

Si vas a renderizar varias tarjetas, necesitas un **array de objetos**, no un único objeto.

8.3. Mezclar toda la lógica en una sola función enorme

Si una única función selecciona nodos, crea tarjetas, decide clases, recorre el array y además hace otras tareas, el código se vuelve más difícil de mantener.

8.4. Abusar de `innerHTML`

Aunque pueda parecer más rápido, en esta fase interesa practicar una construcción más controlada con `createElement()`, `textContent()` y `append()`.

8.5. No vaciar el contenedor si vuelves a renderizar

Aunque en esta fase probablemente renderices solo una vez, conviene empezar a pensar correctamente el ciclo de pintado.

8.6. Dar nombres poco claros a las funciones

Evita nombres como:

- `a`
- `data1`
- `cosa`
- `x`

8.7. Confundir modelado de datos con detalles puramente visuales

No todo lo que ves en la tarjeta tiene que convertirse en una propiedad del objeto. Debes distinguir entre:

- los datos realmente necesarios para construir la tarjeta;
 - y los aspectos decorativos que debe resolver el CSS.
-

9. Entregable esperado

Debes entregar una versión funcional de Mundo Pokémon en la que:

- el listado no esté escrito manualmente en HTML;
 - las tarjetas se generen desde un array local de Pokémon;
 - la estructura esté construida con JavaScript;
 - exista una organización mínima entre datos y renderizado.
-

10. Cierre de la fase

Lo importante en esta fase

Si consigues completar esta parte, habrás dado un paso fundamental en el proyecto: pasar de una maqueta estática a una interfaz generada desde datos. Esa idea será la base de todo lo que venga después. Más adelante cambiará el origen de los datos, aparecerán interacciones y llegarán las APIs, pero el núcleo conceptual seguirá siendo el mismo:

una colección de información puede transformarse en una interfaz dinámica.